Grenoble INP - Phelma

Projet n°32
Année universitaire 2025-2026

Rapport Projet 32

Système d'acquisition / adaptation / conversion des signaux d'un recepteur de modelisme pour une utilisation avec des cartes de programmation (STM32, Arduino, Raspberry Pi) et Conception d'un système de commande Infra Rouge pour vidéo projecteur

Membres du groupe : Corentin PILLEUX Corentin ELETTI Amaury SIMONOT Mathis MENAGER Martin FLEURY Aurélien CAPRON Reda LOUNIS Simon DANQUIGNY Jules ANCEL
Tuteur : Julien TRAVEAUX

27 mai 2026

Table des matières

1 Introduction **4**

Glossaire **5**

2 Sous-projet 1 **5**

2.1 Contextualisation 5
2.2 Travail préparatoire 7
2.2.1 Cahier des charges 7
2.2.2 Diagramme de Gantt 8
2.3 Résolution du problème 8
2.3.1 Protocole 8
2.3.2 Circuit éléctrique 9
2.3.3 Budget 10
2.3.4 Acquisition des signaux 10
2.3.5 Code 12
2.3.6 Résultats 13
2.4 Mise en forme 13
2.4.1 Boitier imprimé en 3D 14
2.4.2 Boitier en bois 14

3 Sous-Projet 2 **16**

3.1 Contextualisation 16
3.2 Contexte et protocoles de radiocommande 16
3.2.1 Le protocole PWM appliqué aux servomoteurs 16
3.2.2 Le protocole S.Bus 16
3.3 Architecture du système 17
3.3.1 Choix du microcontrôleur 17
3.3.2 Architecture fonctionnelle 17
3.3.3 Attribution des broches GPIO 17
3.4 Implémentation logicielle 17
3.4.1 Acquisition des signaux PWM par interruptions 17
3.4.2 Redistribution PWM miroir 19
3.4.3 Encodage et transmission S.Bus 19
3.5 Résultats et validation 20
3.5.1 Banc de test 20
3.5.2 Validation de l'acquisition et de la redistribution 20
3.5.3 Analyse de la trame S.Bus à l'oscilloscope 20
3.6 Conclusion 20

4 Utilisation de l'IA **24**
4.1 Rôle et philosophie d'utilisation 24
4.2 Méthodologie et cas d'usage 24

A Annexes **27**
A.1 Annexe 1 : Modélisation 3D du boîtier (Option abandonnée) 27
A.2 Annexe 2 : Plan de découpe laser du boîtier en bois (Option retenue) 28

Remerciements

Nous remercions Julien Traveaux pour son implication et sa réactivité dans notre projet. A chaque question ainsi qu'a chaque problème il a été là pour nous aiguiller, le tout de manière extrêmement réactive.

Nous remercions Sophie Cornu pour sa présence quasi continue pour répondre à nos questions sur le matériel, et pour avoir géré la commande du matériel dont nous avions besoin.

Nous remercions également le FabLab avec tous les membres qui le composent pour leur aide et pour la mise à disposition de leur matériel. Une pensée également aux responsables des impressions des PCB sans quoi nous n'aurions rien.

Table des figures

1 Cahier des charges du projet 1 7
2 Diagramme de GANTT du projet 1 8
3 Nomenclature des composants matériels de la carte de contrôle 10
4 Principe de transmission temporelle du protocole NEC 11
5 Acquisition de la trame infrarouge à l'oscilloscope 12
6 Boitier réalisé 14
7 Intérieur du boitier 14
10 Architecture fonctionnelle du système. 17
8 Signaux PWM aux positions extrêmes d'un servomoteur (bleu : 1 000 µs ; rouge pointillé : 2 000 µs). La période reste fixe à 20 ms. 18
9 Structure d'une trame S.Bus de 25 octets. 18
11 Vue macroscopique du signal S.Bus (GPIO 0). Les trames se succèdent à intervalles réguliers de 9 ms, conformément aux spécifications Futaba. 21
12 Détail d'une sous-séquence de la trame S.Bus. Chaque bit dure exactement 10 µs (100 000 bauds). La ligne est maintenue au niveau bas au repos, confirmant l'inversion logique caractéristique du protocole. 21
13 Modélisation 3D de la base du boîtier avec les perçages prévus pour les composants 27
14 Modélisation 3D du couvercle du boîtier 27
15 Esquisse 2D (mise à plat) préparée pour la découpe laser du boîtier en bois . . . 28

Liste des tableaux

1 Caractéristiques du protocole S.Bus. 18
2 Attribution des broches GPIO du Raspberry Pi Pico. 18
3 Valeurs PWM mesurées pour trois positions du testeur. 21

1 Introduction

Ce projet est composé de deux parties totalement différentes abordées par 2 sous-groupes constitué au début du projet global.

Pour la première partie, il s'agit de répondre à une demande de Phelma concernant la salle des conseils. En effet après l'utilisation des vidéoprojecteurs lors de diverses réunions, ils restent allumés longtemps après la fin de celles-ci. Cela entraine une sur-utilisation de la lampe et la dégrade de façon inutile impliquant de la changer beaucoup plus régulièrement. L'objectif est alors de construire un boitier qui permet de controller l'état des vidéoprojecteurs pour ainsi palier au problème rencontré.

Pour la deuxième partie, on s'interesse aux signaux. Les systèmes embarqués dédiés à la radiocommande reposent sur des protocoles de transmission de commandes entre un opérateur et des actionneurs. L'évolution de ces protocoles, du signal analogique vers le numérique multiplexé, impose la conception d'interfaces de conversion adaptées. Ce projet vise à réaliser une telle interface sur microcontrôleur Raspberry Pi Pico, capable d'acquérir des signaux de radiocommande, de les redistribuer vers des actionneurs locaux et de les encoder au format numérique S.Bus.

Le document présente d'abord le contexte protocolaire, puis l'architecture matérielle retenue. L'implémentation logicielle est ensuite détaillée, avant la validation expérimentale des résultats.

Glossaire

CPPM (*Compound Pulse Position Modulation*) Protocole de radiocommande multiplexant plusieurs voies PWM sur un seul fil de façon séquentielle, séparées par une impulsion de synchronisation.

GPIO (*General Purpose Input/Output*) Broche d'entrée/sortie numérique à usage général d'un microcontrôleur, configurable par logiciel.

Interruption Mécanisme matériel déclenchant automatiquement l'exécution d'une fonction de traitement (*callback*) en réponse à un événement physique, sans intervention de la boucle principale.

PWM (*Pulse Width Modulation*) Modulation de Largeur d'Impulsion. Encode une consigne de position via la durée d'une impulsion électrique périodique, comprise entre 1 000 et 2 000 µs pour les servomoteurs de radiocommande.

S.Bus Protocole numérique série développé par Futaba, transmettant 16 voies multiplexées sur un unique fil à 100 000 bauds avec logique électrique inversée.

Slice PWM Unité matérielle interne du Raspberry Pi Pico gérant deux canaux PWM indépendants. Chaque *slice* possède sa propre horloge et sa propre période.

UART (*Universal Asynchronous Receiver-Transmitter*) Émetteur-récepteur asynchrone universel assurant la communication série entre composants.

2 Sous-projet 1

2.1 Contextualisation

La gestion de l'efficacité énergétique et l'automatisation des infrastructures sont devenues des standards incontournables dans l'exploitation des bâtiments modernes. Plus particulièrement, au sein des espaces partagés tels que la salle des conseils, les équipements audiovisuels comme les vidéoprojecteurs sont intensivement sollicités au quotidien. Cependant, il est très fréquent que les intervenants omettent d'éteindre ces appareils à l'issue de leurs réunions, reposant ainsi la gestion de l'énergie sur une intervention humaine faillible, ce qui engendre une consommation électrique superflue et une dégradation prématurée du matériel.

Pour résoudre cette problématique, l'objectif de ce projet est de concevoir et de déployer un système d'extinction des vidéoprojecteurs totalement automatique. L'approche retenue repose sur le développement d'un boîtier de contrôle intelligent architecturé autour d'un microcontrôleur Raspberry. Ce dispositif permet de détecter l'état de fonctionnement de deux projecteurs via la tension de leurs ports VGA et de surveiller l'occupation de la salle grâce à un détecteur de présence ainsi qu'un détecteur sonore.

Afin d'exposer clairement le travail réalisé, ce rapport présentera dans un premier temps le cahier des charges matériel, incluant la sélection des composants tels que les émetteurs infrarouges, les transformateurs et les capteurs. Dans un second temps, l'architecture du système et la logique de l'algorithme de contrôle seront détaillées. Enfin, la dernière section abordera l'intégration du boîtier dans le faux plafond et les tests des différentes conditions de déclenchement.

2.2 Travail préparatoire

2.2.1 Cahier des charges

Consigne : Concevoir et déployer un système embarqué autonome capable de se substituer à la télécommande d'origine pour forcer l'extinction des vidéoprojecteurs lorsque la salle des conseils n'est plus utilisée, sans nécessiter d'intervention humaine.

Choix de l'équipe : Pour répondre au mieux à la consigne, nous avons mis en place le diagramme des exigences suivant qui utilise le principe de tiroirs que nous avons trouvé très pratique lors des BE que nous avons effectués cette année :

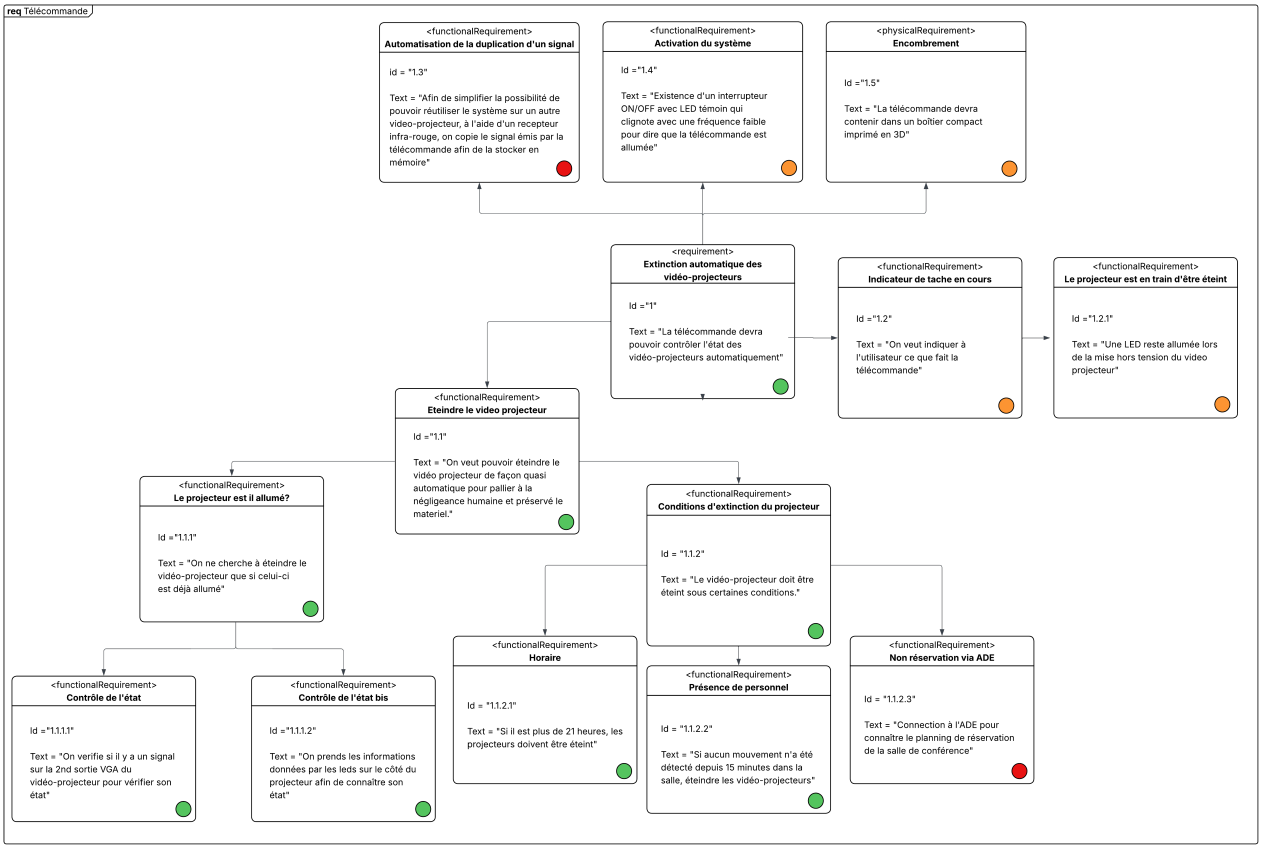


FIGURE 1 – Cahier des charges du projet 1

Tiroir 1 (Ronds Verts) Il est composé des actions basiques nécéssaires pour que notre télécommande soit opérationnelle.

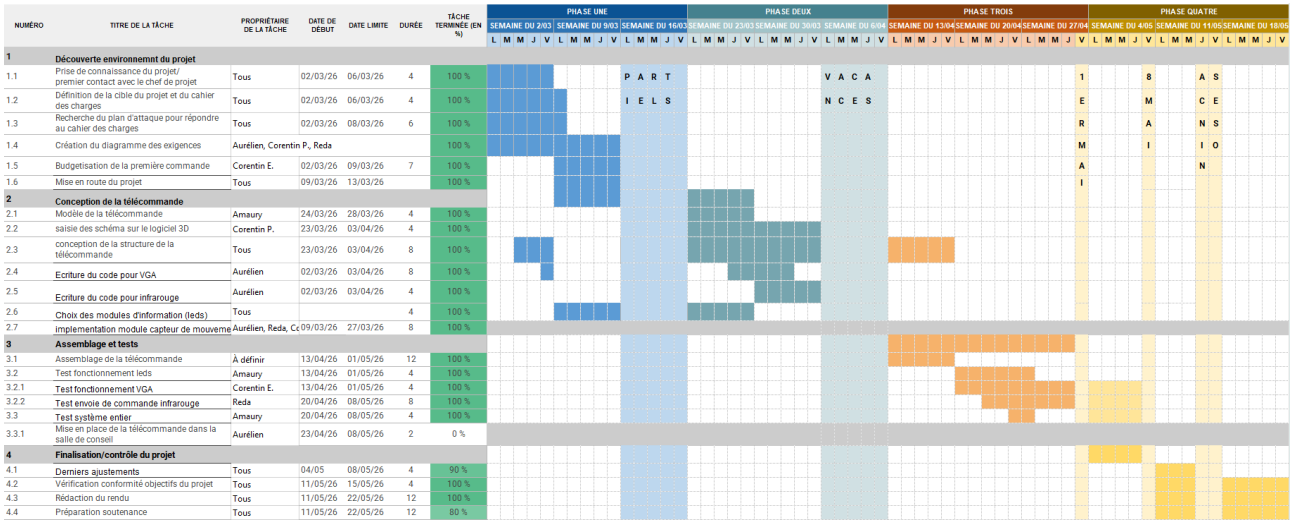
Tiroir 2 (Ronds Oranges) Il contient en son sein des fonctionnaités non indispensables mais qui permettent d'avoir un rendu agréable et d'obtenir des informations sur le fonctionnement de la télécommande.

Tiroir 3 (Ronds Rouges) Il est constitué de paramètres bien plus délicats à incorporer dans le système. Notamment le fait d'éteindre le vidéoprojecteur si la salle n'est pas réservée, qui nécéssiterait un accès privé à l'ADE.

2.2.2 Diagramme de Gantt

Prévision : Afin de pouvoir terminer le projet dans les temps et respecter nos propres échéances, nous avons, en début de parcours, élaboré un diagramme de Gantt afin de nous fixer des objectifs. Depuis, nous avons vu nos tâches accomplies augmenter et nos tâches à accomplir diminuer, et nous avons aujourd'hui presque totalement finalisé notre projet. Seul bémol : nous n'avons pas pu tester notre projet directement dans son environnement final pour le moment.

Ce diagramme nous a principalement permis de garder à l'esprit les objectifs que nous nous étions donnés et ainsi de maintenir une efficacité au travail importante. Cela nous a été bénéfique puisque nous avons pu réaliser l'entièreté des deux premiers tiroirs que nous avions imaginés, ne nous laissant comme seule amélioration que la fin du tiroir numéro 3..



NUMÉRO	TITRE DE LA TÂCHE	PROPRIÉTAIRE DE LA TÂCHE	DATE DE DÉBUT	DATE LIMITE	DURÉE	TÂCHE TERMINÉE (EN %)
1	**Découverte environnemnt du projet**					
1.1	Prise de connaissance du projet/ premier contact avec le chef de projet	Tous	02/03/26	06/03/26	4	100 %
1.2	Définition de la cible du projet et du cahier des charges	Tous	02/03/26	06/03/26	4	100 %
1.3	Recherche du plan d'attaque pour répondre au cahier des charges	Tous	02/03/26	08/03/26	6	100 %
1.4	Création du diagramme des exigences	Aurélien, Corentin P., Reda				100 %
1.5	Budgetisation de la première commande	Corentin E.	02/03/26	09/03/26	7	100 %
1.6	Mise en route du projet	Tous	09/03/26	13/03/26		100 %
2	**Conception de la télécommande**					
2.1	Modèle de la télécommande	Amaury	24/03/26	28/03/26	4	100 %
2.2	saisie des schéma sur le logiciel 3D	Corentin P.	23/03/26	03/04/26	4	100 %
2.3	conception de la structure de la télécommande	Tous	23/03/26	03/04/26	8	100 %
2.4	Ecriture du code pour VGA	Aurélien	02/03/26	03/04/26	8	100 %
2.5	Ecriture du code pour infrarouge	Aurélien	02/03/26	03/04/26	4	100 %
2.6	Choix des modules d'information (leds)	Tous			4	100 %
2.7	implementation module capteur de mouveme	Aurélien, Reda, Cc	09/03/26	27/03/26	8	100 %
3	**Assemblage et tests**					
3.1	Assemblage de la télécommande	À définir	13/04/26	01/05/26	12	100 %
3.2	Test fonctionnement leds	Amaury	13/04/26	01/05/26	4	100 %
3.2.1	Test fonctionnement VGA	Corentin E.	13/04/26	01/05/26	4	100 %
3.2.2	Test envoie de commande infrarouge	Reda	20/04/26	08/05/26	8	100 %
3.3	Test système entier	Amaury	20/04/26	08/05/26	4	100 %
3.3.1	Mise en place de la télécommande dans la salle de conseil	Aurélien	23/04/26	08/05/26	2	0 %
4	**Finalisation/contrôle du projet**					
4.1	Derniers ajustements	Tous	04/05	08/05/26	4	90 %
4.2	Vérification conformité objectifs du projet	Tous	11/05/26	15/05/26	4	100 %
4.3	Rédaction du rendu	Tous	11/05/26	22/05/26	12	100 %
4.4	Préparation soutenance	Tous	11/05/26	22/05/26	12	80 %

FIGURE 2 – Diagramme de GANTT du projet 1

2.3 Résolution du problème

2.3.1 Protocole

Algorithme conditionnel d'extinction des vidéoprojecteurs :

Étape 1 : Acquisition de l'état de P1 (VGA 1) et P2 (VGA 2).

Étape 2 : Vérification des critères d'extinction : Critère horaire : L'heure actuelle est strictement supérieure à 20h00. Critère d'occupation : Le délai depuis le dernier mouvement détecté par C1 est supérieur à 1 heure et 30 minutes.

Étape 3 : Exécution conditionnelle :
Si (Critère horaire VRAI) OU (Critère d'occupation VRAI) :
Si P1 est ALLUMÉ → Envoyer la fréquence f via E1.
Si P2 est ALLUMÉ → Envoyer la fréquence f via E2.

L'algorithme conditionnel d'extinction des vidéoprojecteurs présente les étapes successives du code embarqué. On y observe que le déclenchement des actionneurs infrarouges (Étape 3) est conditionné par la validation d'au moins un des deux critères temporels (Étape 2), mais aussi par la confirmation que le projecteur visé est bel et bien allumé.

2.3.2 Circuit éléctrique

Pour répondre aux contraintes d'autonomie et de discrétion du cahier des charges, l'approche retenue s'articule autour d'un microcontrôleur Arduino Uno (référence A000066) agissant comme unité centrale de traitement. Le protocole d'acquisition de l'état des projecteurs repose sur un câblage direct des ports VGA : le fil vert est assigné à la masse (GND) et le fil noir est connecté aux entrées analogiques (A2 et A3) de l'Arduino pour y lire la sortie vidéo. La détection de présence humaine est assurée par le branchement de deux modules infrarouges passifs (PIR1 et PIR2) et de quatre photorésistances pour évaluer la luminosité ambiante. Enfin, la partie commande est réalisée par l'intégration de deux modules émetteurs infrarouges (GT1087) orientés vers les récepteurs des projecteurs cibles.

2.3.3 Budget

L'ensemble des éléments choisis pour constituer ce circuit est synthétisé dans la figure 3.

Listing du matériel :

Materiel GoTronic :

Composant	Code GOTronic	Quantité	Prix Unitaire TTC	Total
Carte Raspberry Pi Pico 2WH SC1634	39295	1	10.95	10.95€
Détecteur sonore SEN-14262	39461	1	16.80	16.80€
Détecteur de présence mmWave Gravity SEN0609	39143	1	16.70	16.70€
Détecteur de mouvement PIR SBC-PIR	37072	1	6.90	6.90€
Photorésistance PGM1201	03136	4	1.80	7.20€
Module emetteur IR GT1087	35228	2	2.4	4.8€
Module recepteur IR GT1089	35229	1	2.4	2.4€
Bouton-poussoir R1381R	07099	1	1.5	1.5€
Total				**67.25€**

FIGURE 3 – Nomenclature des composants matériels de la carte de contrôle

Le tableau de la figure 3 montre la liste des composants électroniques requis pour le montage, accompagnés de leurs références fournisseurs (Go Tronic, Farnell, RS Components) et de leurs rôles. On observe que le budget global s'élève à 67.25 euros, dont la majorité est allouée à la carte microcontrôleur et aux capteurs que nous utilisons pour déterminer la présence ou non d'une personne dans la salle.

2.3.4 Acquisition des signaux

Pour que notre boîtier puisse se substituer à la télécommande d'origine et forcer l'extinction des vidéoprojecteurs, il est impératif de reproduire fidèlement le signal infrarouge de commande. La première étape a donc consisté à acquérir et déchiffrer ce signal.

En analysant la transmission, nous avons identifié que la télécommande communique avec le vidéoprojecteur en utilisant le **protocole NEC**, un standard très répandu dans les systèmes de transmission infrarouge.

Le principe du protocole NEC Ce protocole repose sur un codage par espacement des impulsions (*Pulse Distance Encoding*) émise sur une fréquence porteuse, généralement de 38 kHz. La distinction entre un bit à l'état logique "0" et un bit à l'état logique "1" ne se fait pas sur la durée de l'impulsion (qui reste constante), mais sur la durée du silence qui la suit :

- **Le "0" logique :** Il est constitué d'une impulsion (*pulse*) de 562 µs suivie d'un espace de 562 µs, soit une durée totale de 1,125 ms.
- **Le "1" logique :** Il est constitué d'une impulsion de 562 µs suivie d'un espace beaucoup plus long de 1,675 ms, soit une durée totale de 2,25 ms.

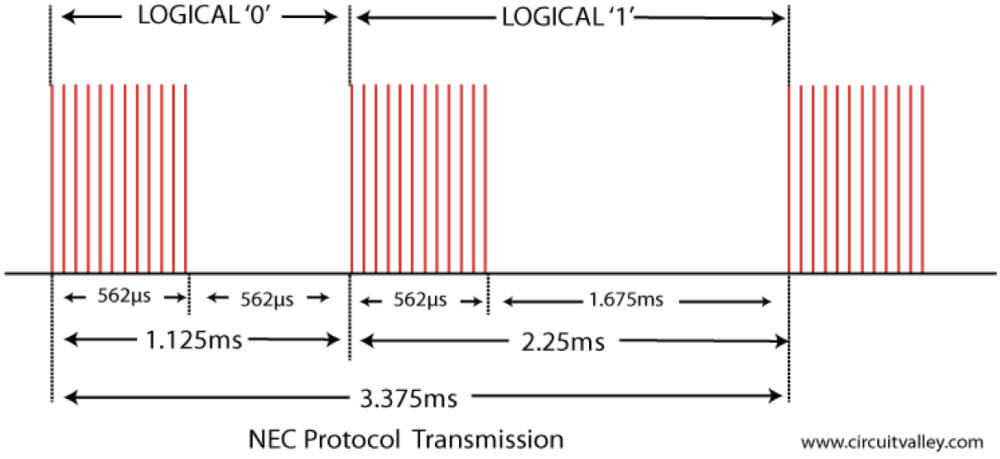


FIGURE 4 – Principe de transmission temporelle du protocole NEC

Déchiffrage à l'aide de l'oscilloscope Pour extraire le code précis d'extinction du vidéoprojecteur, nous avons pointé la télécommande vers un récepteur infrarouge relié à un oscilloscope. La capture du signal (visible sur la figure 5) nous montre la structure complète de la trame de communication.

Sur l'écran de l'oscilloscope, on observe d'abord un très long signal d'en-tête (le *start burst* ou *AGC burst*, typiquement de 9 ms suivi d'un espace de 4,5 ms) qui sert à réveiller et synchroniser le récepteur. Ensuite, on distingue très clairement le train d'impulsions correspondant aux données.

En mesurant l'espacement entre chaque front directement sur l'oscilloscope, nous avons pu identifier visuellement la succession des états hauts et des états bas.

Conversion en code hexadécimal Une trame NEC standard est composée de 32 bits (8 bits pour l'adresse de l'appareil, 8 bits pour l'adresse inversée, 8 bits pour la commande, et 8 bits pour la commande inversée afin de vérifier l'intégrité du message).

En traduisant chaque intervalle court par un **0** et chaque intervalle long par un **1**, nous avons reconstitué la séquence binaire complète de la commande d'extinction. Cette séquence binaire a ensuite été convertie en une valeur **hexadécimale**. C'est ce code hexadécimal final qui

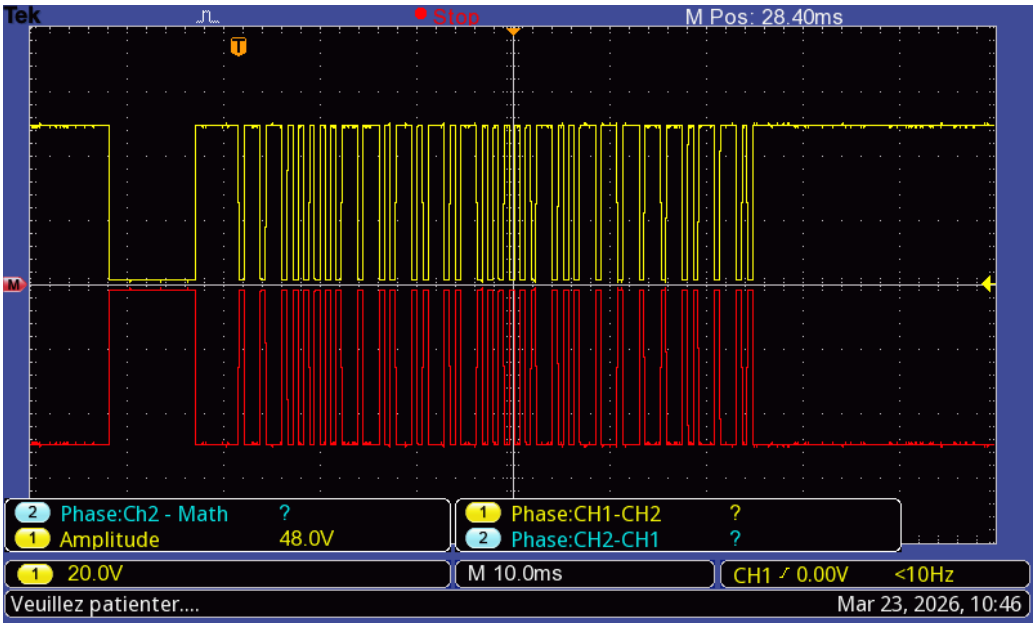


FIGURE 5 – Acquisition de la trame infrarouge à l'oscilloscope

a été implémenté dans notre programme Arduino pour ordonner à nos émetteurs infrarouges (GT1087) de cloner le signal d'origine.

2.3.5 Code

Plutôt que d'utiliser une bibliothèque de haut niveau préexistante pour gérer la communication infrarouge, le parti a été pris de coder manuellement la génération du protocole NEC. Cette approche "bas niveau" permet de s'assurer d'un contrôle total sur les chronogrammes de transmission et valide notre compréhension de la trame acquise précédemment à l'oscilloscope.

L'architecture logicielle de la partie émission s'articule autour de la création de fonctions spécifiques gérant les temps de modulation à 38 kHz (`mark`) et de silence (`space`) en microsecondes. Le fonctionnement se décompose ainsi :

- **Génération de l'en-tête (Header) :** Dans la boucle principale, l'émission commence par le signal de réveil propre au protocole NEC. Comme identifié lors de l'acquisition, il est généré manuellement par la commande `mark(9000)` (impulsion de 9 ms) suivie d'un long silence `space(4500)` (4,5 ms).
- **Transmission de la séquence de 32 bits :** Le code d'extinction déchiffré a été transcrit en une succession stricte d'appels aux fonctions `one()` et `zero()` au sein d'une fonction dédiée nommée `code()`. Chacune de ces sous-fonctions génère la durée spécifique d'impulsion et de silence correspondant à son état logique, reproduisant fidèlement la signature binaire de la télécommande d'origine.
- **Intégration au système global :** Ce bloc de transmission est conditionné par la boucle principale du système (`loop()`). La fonction d'émission n'est appelée que si les conditions d'extinction (délai d'inoccupation de la salle dépassé via les capteurs PIR et

confirmation que le vidéoprojecteur est sous tension via les broches VGA) sont réunies. Une temporisation (`delay`) clôture la trame pour éviter de saturer le récepteur du projecteur.

Note : Un extrait de l'implémentation manuelle du protocole NEC est présenté en Annexe de ce rapport.

2.3.6 Résultats

Afin de valider le fonctionnement du système, plusieurs phases de tests ont été menées de manière incrémentale.

Validation de l'acquisition des états : Les premiers tests ont porté sur la fiabilité des données d'entrée. La détection de l'état du vidéoprojecteur via la tension des broches du port VGA s'est révélée très robuste : le système identifie parfaitement si l'appareil est allumé ou non. Concernant les capteurs de présence (PIR), leur sensibilité a été ajustée pour éviter les faux positifs (comme des variations de lumière), garantissant une détection précise de l'occupation réelle de la salle des conseils.

Validation de la commande infrarouge : L'envoi de la séquence NEC codée manuellement a ensuite été testé en pointant nos émetteurs vers les vidéoprojecteurs. Le clonage du signal s'est avéré 100 % fonctionnel : les modules émetteurs GT1087 parviennent à forcer l'extinction des appareils avec succès. Cela confirme la justesse de notre rétro-ingénierie du signal réalisée à l'oscilloscope et de notre code bas niveau.

Test du système global : Le système complet a été assemblé, câblé et intégré dans son boîtier en bois découpé au laser. Pour les besoins de l'expérience et de la démonstration, les temporisations ont été artificiellement réduites (le délai d'inoccupation de 1 heure et 30 minutes a été ramené à quelques minutes). L'algorithme a parfaitement réagi : en l'absence de mouvement simulé et avec un état VGA lu comme "allumé", le microcontrôleur a correctement déclenché l'envoi de la trame infrarouge, provoquant l'extinction automatique du projecteur. Le cahier des charges initial est donc validé.

2.4 Mise en forme

Une fois le système électronique crée et les codes bons, on a pu réfléchir à mettre en forme le système. On cherche alors à le cacher dans le faux plafond de la salle des conseils. Seulement, notre système est assez sensible à la poussière. On cherche alors à le protéger dans un boitier avec couvercle qui possèdera juste les trous nécéssaires au cablage avec les éléments extérieurs. On doit donc relier entre la carte et l'extérieur au minimum : les pins sur les deux entrées VGA des vidéoprojecteurs, le bouton poussoir qui permet de configurer une nouvelle fréquence de

télécommande, les deux leds qui permettent de connaître l'état de chaque vidéoprojecteur sans avoir à ouvrir la boîte,

Deux options s'offrent à nous, modéliser la boite en 3D puis l'imprimer ou bien en construire une en bois. Sont expliqués juste après les pistes ainsi étudiées et l'option choisie.

2.4.1 Boitier imprimé en 3D

Après quelques mesures pour dimensionner notre boitier on modélise quelque chose de complexe à imprimer en 3D à cause des contraintes d'impression (cf Annexe n°1). On passe rapidement à l'option en bois.

2.4.2 Boitier en bois

On réalise donc un boitier en bois avec couvercle. Le modèle a été trouvé sur la bibliothèque boxes.py, que l'on met à l'échelle en fonction de la taille de nos composants. Avant de passer à la découpe laser on modifie l'esquisse du modèle sur logiciel afin de percer des trous qui seront nécessaires pour relier les différents composants externes au boitier. (cf Annexe n°2). Le résultat final se trouve sur les figures au dessous. Les dimensions de la boîte sont du (14cm x 15.5cm x 10cm). Cela laisse une place confortable pour positionner la plaque à l'intérieur, et faire passer les fils nécessaires.

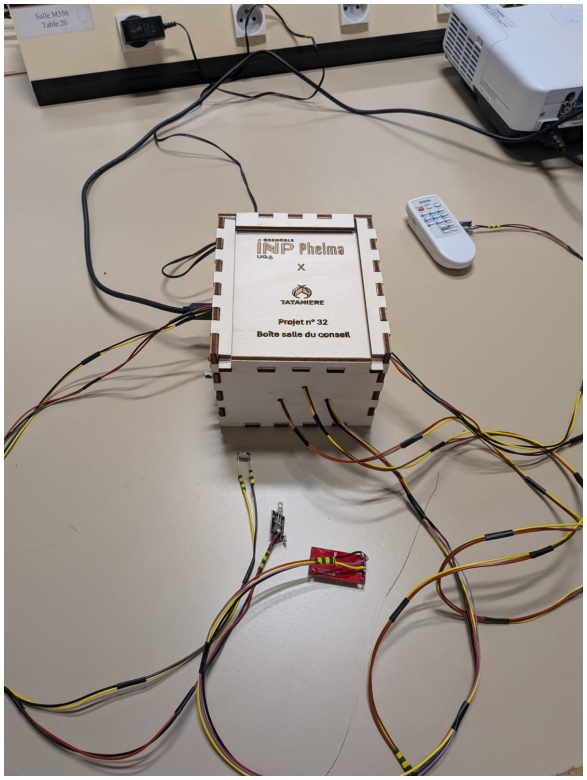


FIGURE 6 – Boitier réalisé



FIGURE 7 – Intérieur du boitier

Résumé

Ce projet résout un problème récurrent dans la salle des conseils de Phelma : les vidéoprojecteurs qui restent allumés toute la nuit. L'objectif était de concevoir un boîtier de contrôle intelligent caché dans le plafond. Ce boîtier se connecte directement à des capteurs de mouvement et de bruit installés dans la salle. Une carte de traitement électronique surveille constamment l'environnement. Si aucun mouvement ni bruit n'est détecté pendant 30 minutes, ou s'il est plus de 21h, le système agit automatiquement. Il utilise des émetteurs infrarouges pour envoyer une commande d'extinction aux vidéoprojecteurs, agissant exactement comme une télécommande pour les éteindre et améliorer leur durée de vie en empechant la lampe de trop se dégrader.

Abstract

This project solves a common problem in the Phelma boardroom : video projectors left turned on all night. The goal was to design an intelligent control box hidden inside the ceiling. This box connects directly to motion and sound sensors installed around the room. An electronic processing board constantly monitors the environment. If no movement or noise is detected for 30 minutes, or if it is past 9 :00 PM, the system automatically takes action. It uses infrared emitters to send a shutdown command to the projectors, acting exactly like a remote control to turn them off and improve their durability.

Keywords : Video projector, automatic, sensors, monitoring, remote control, durability electronic processing, boardroom.

3 Sous-Projet 2

3.1 Contextualisation

Dans les systèmes embarqués dédiés à la radiocommande, la représentation traditionnelle des ordres de commande repose sur le protocole PWM (Modulation de Largeur d'Impulsion), où la durée d'une impulsion électrique encode la position souhaitée d'un actionneur. Ce protocole, bien que simple et robuste, présente un inconvénient structurel majeur : il requiert un fil physique distinct par voie de commande, ce qui complexifie le câblage dans les systèmes multi-actionneurs.

Face à cette contrainte, des protocoles numériques multiplexés ont émergé, parmi lesquels le S.Bus de Futaba, qui transmet jusqu'à 16 voies sur un unique fil de données à 100 000 bauds. L'adoption de ces protocoles impose la disponibilité d'une interface capable de convertir des signaux PWM analogiques vers ces formats numériques, tout en maintenant une redistribution directe des signaux d'entrée vers les actionneurs locaux.

L'objectif de ce projet est de concevoir et d'implémenter une telle interface sur un microcontrôleur Raspberry Pi Pico. Le système doit simultanément assurer trois fonctions : l'acquisition de signaux PWM en entrée par interruptions GPIO, la redistribution de ces signaux vers des servomoteurs de sortie, et l'encodage de l'ensemble dans une trame S.Bus numérique.

Le document présente d'abord le contexte protocolaire (section 3.2), puis l'architecture matérielle retenue (section 3.3). L'implémentation logicielle est ensuite détaillée (section 3.4), avant la validation expérimentale des résultats (section 3.5).

3.2 Contexte et protocoles de radiocommande

3.2.1 Le protocole PWM appliqué aux servomoteurs

Le protocole PWM en radiocommande repose sur l'émission périodique d'impulsions à 50 Hz (période de 20 ms). La position angulaire θ du servomoteur est encodée de façon linéaire par la durée de l'impulsion τ, selon la relation :

$$\theta = \theta_{\min} + \frac{\tau - \tau_{\min}}{\tau_{\max} - \tau_{\min}} \cdot (\theta_{\max} - \theta_{\min}) \tag{1}$$

avec $\tau_{\min} = 1\,000$ µs, $\tau_{\max} = 2\,000$ µs et $\tau_{\text{neutre}} = 1\,500$ µs. La figure 8 illustre les deux cas extrêmes.

3.2.2 Le protocole S.Bus

Le S.Bus est un protocole numérique série développé par Futaba, transmettant 16 voies multiplexées sur un unique fil. Le tableau 1 en résume les caractéristiques principales.

La structure de la trame est présentée en figure 9. La conversion d'une durée PWM τ en valeur S.Bus v suit une loi linéaire analogue à l'équation 1, avec $v_{\min} = 172$, $v_{\max} = 1\,811$.

La connaissance de ces deux protocoles justifie les choix architecturaux décrits dans la section suivante.

3.3 Architecture du système

3.3.1 Choix du microcontrôleur

Le Raspberry Pi Pico, basé sur le microcontrôleur RP2040 , a été retenu pour trois raisons principales. Son module PWM matériel permet de générer des signaux servo précis sans solliciter le processeur. Ses broches GPIO supportent les interruptions sur front montant et descendant, indispensables pour la mesure de durées d'impulsions à la microseconde.Enfin, son UART matériel supporte le format 100 000 bauds avec parité paire, conformément aux spécifications S.Bus.

3.3.2 Architecture fonctionnelle

Le système assure trois fonctions simultanées, schématisées en figure 10 : acquisition des signaux PWM d'entrée, redistribution miroir vers les servomoteurs, et encodage S.Bus sur un fil unique.

FIGURE 10 – Architecture fonctionnelle du système.

3.3.3 Attribution des broches GPIO

Le tableau 2 récapitule l'attribution des broches. Les sorties servo ont été volontairement affectées aux broches 6, 9 et 11, appartenant respectivement aux *slices* PWM 3, 4 et 5 du RP2040. Cette répartition garantit l'indépendance complète des trois signaux générés : deux broches partageant un même *slice* ne peuvent produire des signaux de périodes différentes simultanément.

L'architecture matérielle étant définie, la section suivante détaille l'implémentation logicielle de chacune des trois fonctions.

3.4 Implémentation logicielle

L'implémentation repose sur trois modules asynchrones et indépendants : l'acquisition par interruptions, la redistribution PWM et l'encodage S.Bus.

3.4.1 Acquisition des signaux PWM par interruptions

Motivation. La mesure de la durée d'une impulsion PWM requiert une précision à la microseconde. Une approche par scrutation (*polling*) introduirait une latence indéterministe

[->] (0,0) – (10,0) node[right] t (ms) ; [->] (0,0) – (0,2.5) node[above] Tension (V) ; [thick, blue] (0,0)–(0,2)–(0.5,2)–(0.5,0)–(5,0

FIGURE 8 – Signaux PWM aux positions extrêmes d'un servomoteur (bleu : 1 000 µs ; rouge pointillé : 2 000 µs). La période reste fixe à 20 ms.

TABLE 1 – Caractéristiques du protocole S.Bus.

Paramètre	**Valeur**
Débit	100 000 bauds
Format	8 bits, parité paire, 2 bits de stop (8E2)
Logique	Inversée (0 V = état haut)
Nombre de voies	16 voies sur 11 bits
Période de trame	9 ms (≈ 111 Hz)
Longueur de trame	25 octets
Plage de valeurs	172 (mini) – 1 811 (maxi), neutre : 1 023

[fill=headerblue!20] (0,0) rectangle (1,0.7) node[midway]0x0F ; [fill=lightgray] (1,0) rectangle (8,0.7) node[midway]22 octets

FIGURE 9 – Structure d'une trame S.Bus de 25 octets.

TABLE 2 – Attribution des broches GPIO du Raspberry Pi Pico.

GPIO	**Direction**	**Fonction**	**Rôle**
2	Entrée	Interruption	Signal PWM voie 1
3	Entrée	Interruption	Signal PWM voie 2
4	Entrée	Interruption	Signal PWM voie 3
6	Sortie	PWM (*slice* 3)	Servo miroir voie 1
9	Sortie	PWM (*slice* 4)	Servo miroir voie 2
11	Sortie	PWM (*slice* 5)	Servo miroir voie 3
0	Sortie	UART TX	Signal S.Bus (100 000 bauds, 8E2)

incompatible avec cette exigence. L'emploi d'interruptions matérielles garantit un déclenchement en quelques cycles d'horloge.

Protocole. Une fonction de rappel (*callback*) est associée à chaque broche d'entrée. À chaque front montant, l'horodatage courant est mémorisé. À chaque front descendant, la durée de l'impulsion est calculée et filtrée dans la plage valide [1 000 ; 2 000] µs.

[language=C, caption=Fonction de rappel pour l'acquisition sur la broche GPIO 2 (voie 1).]
void rappel$_voie_0()uint32_t t = micros(); if(digitalRead(ENTREES[0]) == HIGH)debut_impulsion[0] = t; //fro

Interprétation. Les variables `valeurs_pwm[]` sont déclarées `volatile` afin d'interdire toute mise en cache par le compilateur, garantissant la cohérence des données entre la *callback* et la boucle principale.

Conclusion partielle. L'acquisition par interruptions offre une précision de mesure de l'ordre de quelques microsecondes, ce qui est largement suffisant pour le contrôle de servomoteurs RC dont la résolution mécanique ne dépasse pas 500 pas sur la plage 1 000–2 000 µs.

3.4.2 Redistribution PWM miroir

Motivation. La redistribution miroir retransmet chaque signal d'entrée vers un servomoteur de sortie, permettant une vérification visuelle directe de l'acquisition.

Protocole. Le module PWM matériel du Pico est configuré à 50 Hz (période 20 ms, résolution 1 µs). La bibliothèque `Servo` de l'environnement Arduino prend en charge la génération du signal.

[language=C, caption=Configuration et mise à jour des sorties servo dans la boucle principale.]
// Initialisation servos[i].attach(SORTIES[i], PWM$_MIN, PWM_MAX); servos[i].writeMicroseconds(PWM
// Mise a jour (dans loop()) for (int i = 0 ; i < NB$_VOIES; i++)servos[i].writeMicroseconds(valeurs_pwm

Conclusion partielle. Le module PWM matériel génère les signaux de sortie de façon entièrement autonome, sans solliciter le processeur entre deux mises à jour, ce qui libère des ressources de calcul pour l'encodage S.Bus.

3.4.3 Encodage et transmission S.Bus

Motivation. L'encodage S.Bus permet de transmettre l'ensemble des voies sur un unique fil numérique, compatible avec tout récepteur ou contrôleur de vol supportant ce protocole industriel.

Protocole. Chaque valeur PWM est d'abord convertie en valeur S.Bus par interpolation linéaire. Les 16 valeurs sur 11 bits sont ensuite empilées de façon contiguë dans les octets 1 à 22 de la trame, sans alignement sur les frontières d'octets. L'ensemble est enfin inversé bit à bit pour respecter la logique électrique inversée du S.Bus.

[language=C, caption=Encodage des voies et inversion logique de la trame S.Bus.] int bit = 0 ; for (int i = 0 ; i < 16 ; i++) int val = (i < NB$_VOIES)?pwm_vers_sbus(valeurs_pwm[i]) :

$SBUS_MID; for(intb = 0; b < 11; b++)if(val(1 << b))trame[1 + bit/8]| = (1 << (bitbit++; for(inti = 0; i < 25; i++)trame[i] = \ trame[i]; //inversionS.BusSerial1.write(trame, 25);$

Conclusion partielle. La séparation entre l'acquisition asynchrone et l'émission périodique garantit un flux d'exécution stable. Le retard entre réception et redistribution est borné par la période de la boucle principale (9 ms), inférieure à la période servo standard (20 ms).

3.5 Résultats et validation

3.5.1 Banc de test

La validation a été conduite avec un testeur Pro-TroniK ST-V2 (réf. 7971), fournissant un signal PWM variable sur trois sorties simultanées, et un servomoteur Dualsky DS3001. L'environnement de développement utilisé est l'IDE Arduino avec le *core* RP2040 d'Earle Philhower [4].

3.5.2 Validation de l'acquisition et de la redistribution

Les valeurs mesurées via le moniteur série confirment une retranscription fidèle aux trois positions de référence du testeur, comme présenté dans le tableau 3.

Les trois voies affichent des valeurs identiques, ce qui est cohérent avec le comportement du testeur dont les trois sorties sont issues du même générateur interne. Le servomoteur de test suit visuellement et sans à-coups les variations de commande, confirmant une latence négligeable.

3.5.3 Analyse de la trame S.Bus à l'oscilloscope

La figure 11 présente l'allure macroscopique de la trame S.Bus capturée en sortie du microcontrôleur (broche GPIO 0).

Un zoom sur la séquence numérique est présenté en figure 12 afin de valider le débit binaire et l'inversion logique.

L'interprétation de ces relevés confirme : (1) le maintien de la ligne à l'état bas au repos (logique inversée), (2) une durée binaire de 10 µs correspondant aux 100 000 bauds, et (3) une périodicité inter-trames de 9 ms conforme aux spécifications. Ces trois critères valident l'ensemble de la chaîne de conversion.

3.6 Conclusion

Ce projet a permis la conception et la validation d'une interface de conversion PWM vers S.Bus fonctionnelle sur Raspberry Pi Pico. La combinaison de l'acquisition par interruptions asynchrones et de la génération PWM matérielle assure une grande robustesse : les signaux d'entrée sont retranscrits fidèlement vers les actionneurs locaux, et l'encodage numérique S.Bus

TABLE 3 – Valeurs PWM mesurées pour trois positions du testeur.

Position	**V1 (µs)**	**V2 (µs)**	**V3 (µs)**
Gauche (mini)	1 000	1 000	1 000
Centre (neutre)	1 500	1 500	1 500
Droite (maxi)	2 000	2 000	2 000

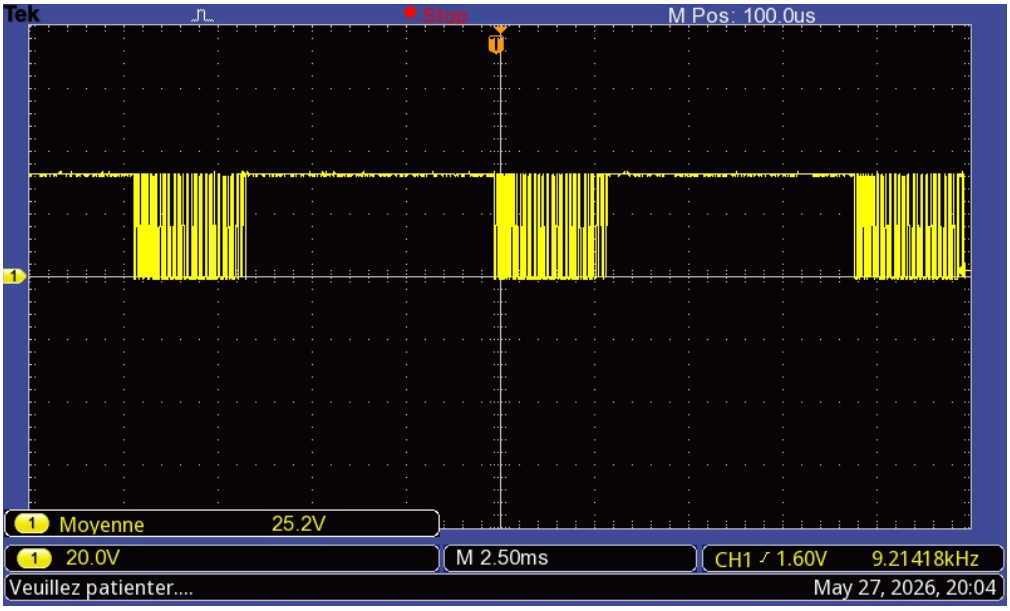


FIGURE 11 – Vue macroscopique du signal S.Bus (GPIO 0). Les trames se succèdent à intervalles réguliers de 9 ms, conformément aux spécifications Futaba.

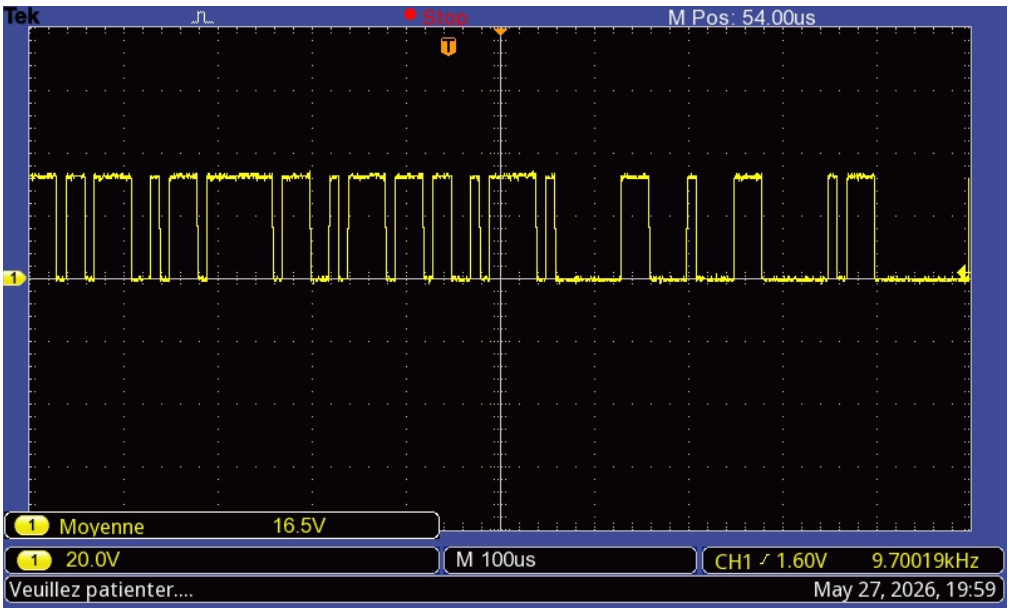


FIGURE 12 – Détail d'une sous-séquence de la trame S.Bus. Chaque bit dure exactement 10 µs (100 000 bauds). La ligne est maintenue au niveau bas au repos, confirmant l'inversion logique caractéristique du protocole.

respecte strictement les spécifications électriques et temporelles du standard industriel Futaba, comme l'attestent les mesures à l'oscilloscope.

Ces résultats démontrent qu'un microcontrôleur à faible coût peut constituer une interface transparente et flexible entre un récepteur de radiocommande et de multiples actionneurs, avec des applications directes en robotique et dans les systèmes de drones.

Plusieurs axes d'amélioration peuvent être envisagés. L'intégration du protocole CPPM en entrée permettrait une interface directe avec la sortie multiplexée d'un récepteur RC standard, supprimant le besoin de six fils d'entrée. Le déploiement d'un module de mixage paramétrable, définissant des lois de combinaison entre voies, ouvrirait enfin des perspectives d'utilisation avancées pour la gestion des liaisons de vol embarquées.

Références

[1] Futaba Corporation, *S.Bus Protocol Specification*, Documentation technique, 2012.

[2] Raspberry Pi Ltd, *Raspberry Pi Pico C/C++ SDK*, `https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf`, consulté le 27 mai 2026.

[3] OpenTX Documentation, *PWM, CPPM, S.Bus – Kesako ?*, `https://opentx-doc.fr/wiki/pwm-cppm-s-bus-kesako-ee`, consulté le 27 mai 2026.

[4] Earle Philhower, *Arduino-Pico – Raspberry Pi Pico Arduino core*, `https://github.com/earlephilhower/arduino-pico`, consulté le 27 mai 2026.

[5] Dualsky, *DS3001 Digital Servo Datasheet*, Documentation produit, 2020.

Résumé

Les systèmes de radiocommande exploitent généralement des signaux PWM nécessitant un câblage complexe. Ce projet présente la conception d'une interface sur Raspberry Pi Pico permettant d'acquérir plusieurs signaux PWM indépendants, de les redistribuer vers des actionneurs, et de les encoder simultanément dans une trame numérique S.Bus multiplexée. La solution exploite la génération matérielle et le décodage par interruptions, traitant jusqu'à six voies indépendantes encodées dans une trame de 16 voies à 100 000 bauds. Les résultats expérimentaux valident la robustesse et la faible latence du système.

Mots-clés : PWM, S.Bus, Raspberry Pi Pico, radiocommande, servomoteur, microcontrôleur, interruptions, multiplexage.

Abstract

Radio-controlled systems typically rely on Pulse Width Modulation (PWM) signals, which require complex wiring. This project introduces a Raspberry Pi Pico-based interface designed to acquire multiple independent PWM signals, redistribute them to actuators, and simultaneously encode them into a multiplexed digital S.Bus frame. By leveraging hardware generation and interrupt-driven decoding, the solution processes up to six independent channels encoded into a 16-channel, 100 000 baud frame. Experimental results validate the system's robustness and low latency.

Keywords : PWM, S.Bus, Raspberry Pi Pico, radio control, servomotor, microcontroller, interrupts, multiplexing.

4 Utilisation de l'IA

4.1 Rôle et philosophie d'utilisation

Il a été établi dès le début du projet qu'une délégation complète du travail à une intelligence artificielle serait contre-productive. Le temps alloué à la réalisation du projet offrait une opportunité d'apprentissage et de formation technique indispensable. De plus, la démarche globale visait à simuler un environnement d'entreprise réaliste, dans lequel la dépendance excessive à des outils automatisés doit rester mesurée, l'évolution de ces technologies en milieu professionnel étant encore sujette à incertitudes.

Par conséquent, l'intégration de l'IA s'est limitée à des rôles d'assistance ciblés. L'outil exclusif retenu pour ces tâches a été Gemini (dans sa version Pro, proposée par Google), dont l'accès a été obtenu en début d'année universitaire. Ce choix a été motivé par la similarité de ses performances avec les autres standards du marché (tels que ChatGPT, Claude ou Mistral) et par une maîtrise préalable de l'outil par les différents membres du groupe.

4.2 Méthodologie et cas d'usage

L'utilisation de cet agent s'est structurée autour de deux axes d'assistance :

L'axe documentaire : Cet aspect englobe la recherche et l'extraction d'informations techniques précises. À titre d'exemple, lors du développement du premier sous-projet, l'IA a été sollicitée pour vérifier si le port VGA du vidéoprojecteur cible émettait un signal mesurable lorsqu'il était sous tension. Cette piste, confirmée par l'IA via l'analyse de fiches techniques, est devenue le principe de détection fondamental du système. De même, pour le second sous-projet, l'IA a grandement facilité la compréhension et l'extraction d'informations au sein des documentations complexes liées aux fonctions matérielles.

L'axe correctionnel : Ce second usage désigne l'assistance au débogage. L'IA a été employée pour pointer rapidement les erreurs (syntaxiques ou logiques) au sein des codes développés. Cette méthode a permis d'isoler et de corriger les dysfonctionnements sans avoir à relire intégralement les scripts, un processus souvent source de frustration et de perte de temps.

Ingénierie de requête (*Prompt Engineering*) : Afin d'obtenir des résultats pertinents et d'éviter que l'IA ne génère des solutions toutes faites, une structure stricte de requête a été mise en place pour interroger le modèle :

- **Contexte du projet :** Explication du but global et des contraintes du cahier des charges.
- **Problématique :** Description de la démarche suivie, de l'objectif immédiat et du blocage technique rencontré, aboutissant à une question précise.

— **Pistes envisagées :** Proposition de débuts de réflexion pour guider l'IA tout en laissant la porte ouverte à de nouvelles solutions.

Exemple de requête formulée lors du projet :

« Le but du projet est de créer un boitier capable d'éteindre sous certaines conditions deux vidéoprojecteurs placés dans une salle des conseils. En effet, après les réunions, les vidéoprojecteurs restent parfois allumés. Cela pose des problèmes au niveau énergétique et risque de griller la lentille par surchauffe.

On souhaite connaître l'état du vidéoprojecteur à un instant t, et on ne sait pas trop comment le savoir. En effet il semble compliqué de coller un capteur qui vérifie l'état grâce aux LED du vidéoprojecteur. Je joins en PDF la doc de notre modèle qui est assez vétuste. As-tu des solutions en réserve pour répondre à notre besoin ?

Tu peux rechercher dans la doc des éléments de réponse, et également sur ce qui est sur le WEB pour savoir si d'autres méthodes existent. On avait pensé à connaître l'état du système via le port USB avec une possible fonctionnalité du système mais nous n'avons pas réussi à trouver d'éléments de doc pour réaliser cela. »

Références

[1] N. Laurent et B. Delaunay, *"La grammaire pour tous"*, Collection Bescherelle, Edition Hatier, 2012[cite : 579].

[2] A.-P. Jauho and L. Reggiani, «Quantum correction to the Monte Carlo solution of hot-electron transport in semiconductors», *Solid State Electron.*, vol. 31, no. 3/4, pp. 535-538, 1988[cite : 595, 596].

A Annexes

A.1 Annexe 1 : Modélisation 3D du boîtier (Option abandonnée)

Cette annexe présente les modélisations 3D initiales du boîtier et de son couvercle. Cette piste a finalement été écartée en raison de la complexité d'impression liée aux contraintes matérielles (notamment les porte-à-faux et les perçages multiples).

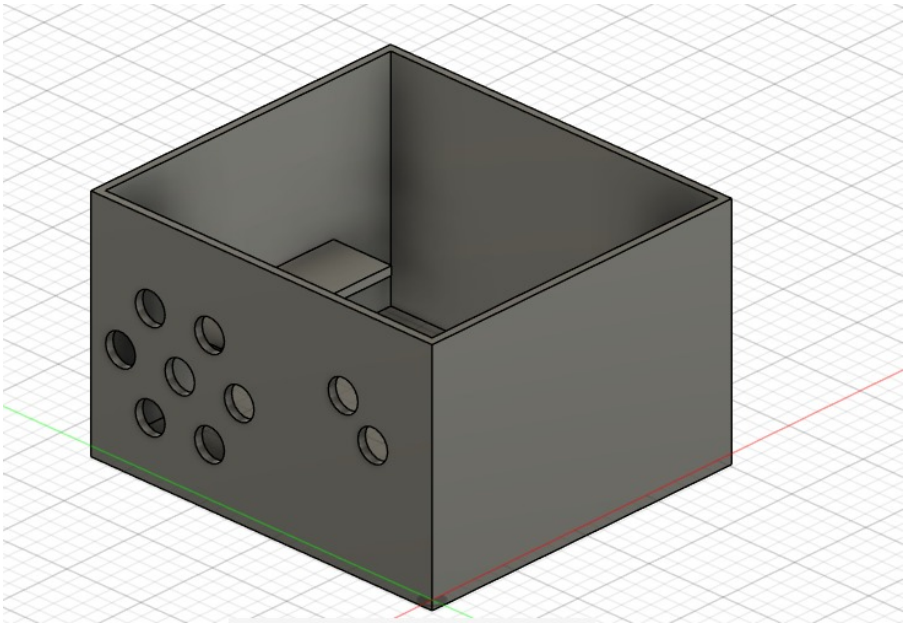


FIGURE 13 – Modélisation 3D de la base du boîtier avec les perçages prévus pour les composants

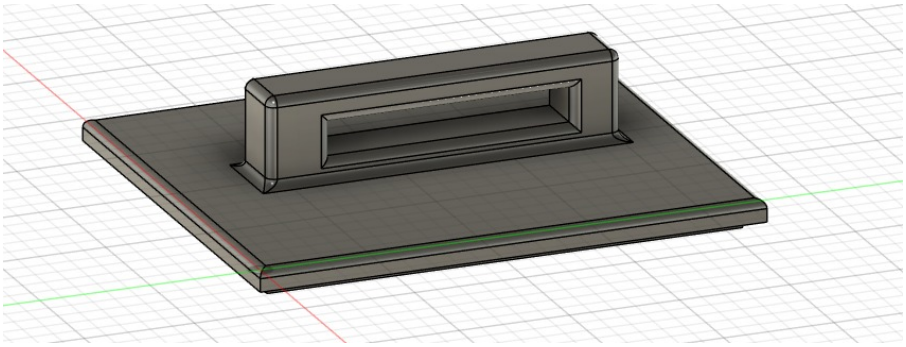


FIGURE 14 – Modélisation 3D du couvercle du boîtier

A.2 Annexe 2 : Plan de découpe laser du boîtier en bois (Option retenue)

Cette annexe présente le plan 2D utilisé pour la découpe laser du boîtier final. Le modèle de base, généré via la bibliothèque *boxes.py*, a été mis à l'échelle et modifié afin d'y intégrer les perçages exacts nécessaires au passage des câbles, aux LED et au bouton poussoir.

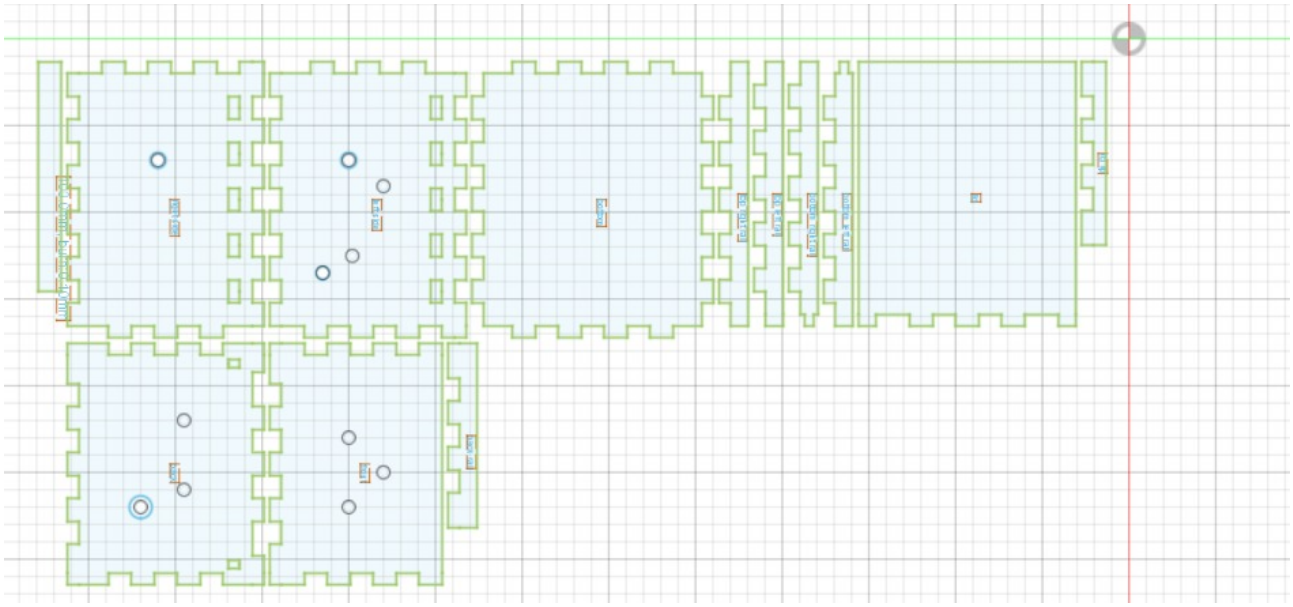


FIGURE 15 – Esquisse 2D (mise à plat) préparée pour la découpe laser du boîtier en bois